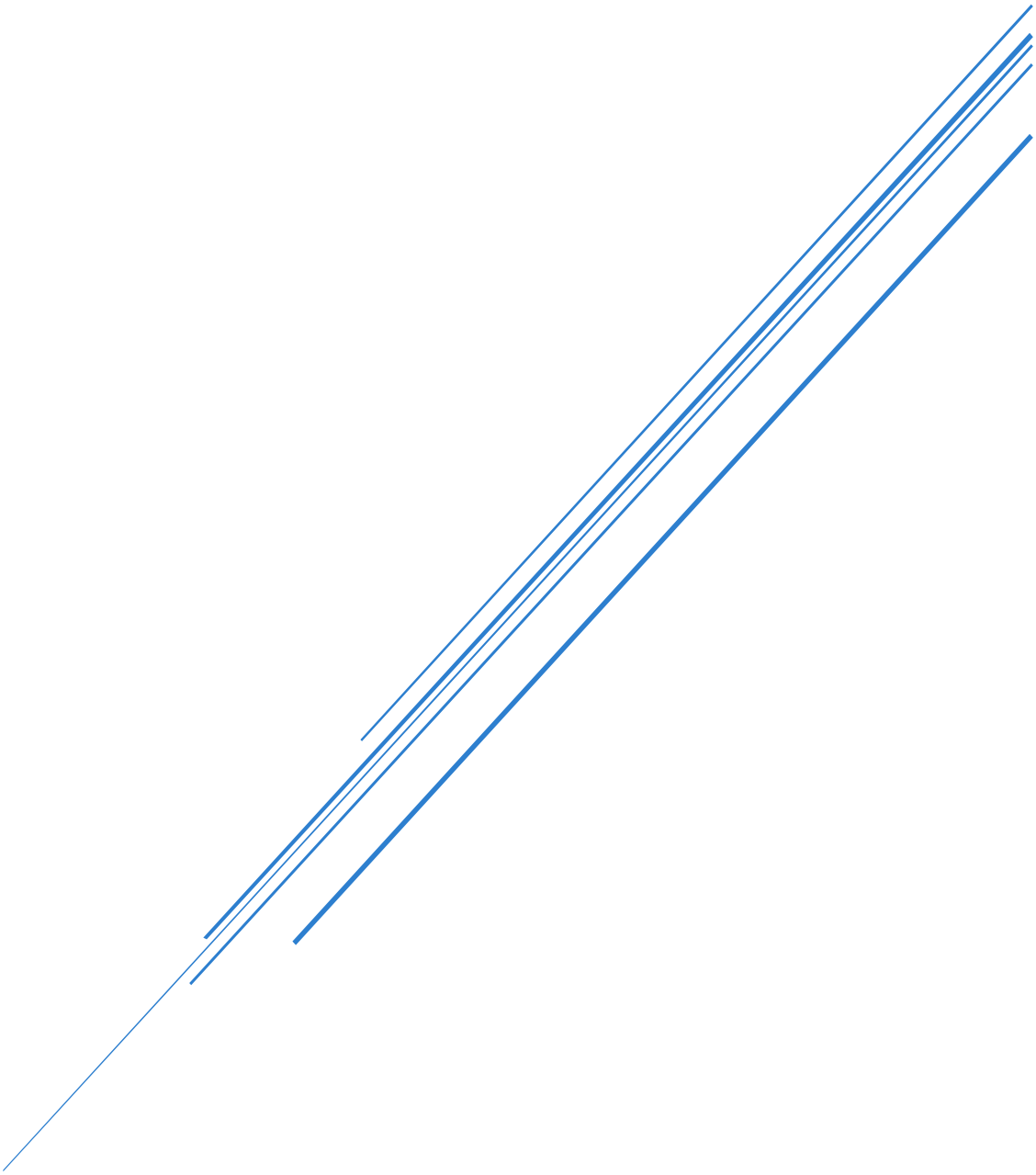
AWS 2 11

Roberto Ripoll Cones

01/11/2026



IES Juan de Garay

CE CLOUD – Administración de redes en la nube

Contenido

1. Python 3
1.1. Creación 3
1.1.1. Creación de la VPC 3
1.1.2. Creación de las subredes y asignación automática de IP publica 3
1.1.3. Creación del grupo de seguridad y reglas SSH e ICMP 4
1.1.4. Creación y asociación del Internet Gateway 5
1.1.5. Creación de la tabla y configuración de la ruta hacia Internet..... 5
1.1.6. Asociación de las subredes a la tabla personalizada 6
1.1.7. Lanzamiento EC2, asignación de IP privada y obtención de IP publica 6
1.1.8. Orden de ejecución 7
1.2. Borrado 8
1.2.1. Búsqueda de VPC por etiquetas 8
1.2.2. Terminación de instancias EC2 9
1.2.3. Eliminar recursos respetando dependencias 10
1.2.4. Función principal de borrado..... 11
1.2.5. Funcionamiento 11
2. Bash 12
2.1. Creación 12
2.1.1. Creación de la VPC y subredes 12
2.1.2. Configuración del grupo de seguridad 12
2.1.3. Creación y asociación de la tabla de enrutamiento..... 13
2.1.4. Creación y asociación del Internet Gateway 13
2.1.5. Creación de instancias EC2 14
2.1.6. Prueba ejecución 14
2.2. Borrado 15
2.2.1. Búsqueda de VPC mediante etiquetas 15
2.2.2. Terminación de instancias EC2 15
2.2.3. Eliminación de tablas de enrutamiento 16
2.2.4. Eliminación de Internet Gateway 17
2.2.5. Gestión de interfaces de red (ENI)..... 18

2.2.6. Función principal de borrado y confirmación..... 19
2.2.7. Ejecución del script..... 20

1. Python

1.1. Creación

1.1.1. Creación de la VPC

```
9   def crear_vpc(ec2):
10      return ec2.create_vpc(
11          CidrBlock='172.16.0.0/16',
12          TagSpecifications=[
13              {
14                  'ResourceType': 'vpc',
15                  'Tags': [
16                      {'Key': 'Name', 'Value': 'VPC-RRC-Python'},
17                      {'Key': 'Owner', 'Value': 'Roberto Ripoll Cones'},
18                      {'Key': 'tipo', 'Value': 'prueba'}
19                      ]
20              }
21          ],
22          InstanceTenancy='default')
```

1.1.2. Creación de las subredes y asignación automática de IP publica

```
27  def crear_subred(ec2, vpc_id, cidrBlock, name):
28      subred = ec2.create_subnet(
29          VpcId=vpc_id,
30          CidrBlock=cidrBlock,
31          TagSpecifications=[
32              {
33                  'ResourceType': 'subnet',
34                  'Tags': [
35                      {'Key': 'Name', 'Value': name},
36                      {'Key': 'Owner', 'Value': 'Roberto Ripoll Cones'},
37                      {'Key': 'tipo', 'Value': 'prueba'}
38                  ]
39              }
40          ])
41      return subred
```

```
67  def habilitar_ip_publica_subred(ec2, subred_id):
68      ec2.modify_subnet_attribute(
69          SubnetId=subred_id,
70          MapPublicIpOnLaunch={'Value': True}
71      )
72
```

1.1.3. Creación del grupo de seguridad y reglas SSH e ICMP

```python
def crear_grupo_seguridad(ec2, vpc_id):
    grupo = ec2.create_security_group(
        GroupName='SG-RRC-Python',
        Description='Grupo de seguridad para la instancia EC2',
        VpcId=vpc_id,
        TagSpecifications=[
            {
                'ResourceType': 'security-group',
                'Tags': [
                    {'Key': 'Name', 'Value': 'SG-RRC-Python'},
                    {'Key': 'Owner', 'Value': 'Roberto Ripoll Cones'},
                    {'Key': 'tipo', 'Value': 'prueba'}
                ]
            }
        ]
    )

    habilitar_reglas_grupo_seguridad(ec2, obtener_grupo_seguridad_id(grupo))

    return grupo

```

```python
def habilitar_reglas_grupo_seguridad(ec2, grupo_id):
    ec2.authorize_security_group_ingress(
        GroupId=grupo_id,
        IpPermissions=[
            {
                'IpProtocol': 'tcp',
                'FromPort': 22,
                'ToPort': 22,
                'IpRanges': [{'CidrIp': '0.0.0.0/0'}]
            },
            {
                'IpProtocol': 'icmp',
                'FromPort': -1,
                'ToPort': -1,
                'IpRanges': [{'CidrIp': '0.0.0.0/0'}]
            }
        ]
    )
```

1.1.4. Creación y asociación del Internet Gateway

```python
def crear_gateway_internet(ec2, vpc_id):
    gateway = ec2.create_internet_gateway(
        TagSpecifications=[
            {
                'ResourceType': 'internet-gateway',
                'Tags': [
                    {'Key': 'Name', 'Value': 'IGW-RRC-Python'},
                    {'Key': 'Owner', 'Value': 'Roberto Ripoll Cones'},
                    {'Key': 'tipo', 'Value': 'prueba'}
                ]
            }
        ]
    )

    asociar_gateway_internet_a_vpc(ec2, obtener_gateway_internet_id(gateway), vpc_id)

    return gateway

```

```python
def asociar_gateway_internet_a_vpc(ec2, gateway_id, vpc_id):
    ec2.attach_internet_gateway(
        InternetGatewayId=gateway_id,
        VpcId=vpc_id
    )
    print(f"Gateway de Internet {gateway_id} asociado a la VPC {vpc_id}")
```

1.1.5. Creación de la tabla y configuración de la ruta hacia Internet

```python
def crear_tablas_enrutamiento(ec2, vpc_id, name):
    tabla = ec2.create_route_table(
        VpcId=vpc_id,
        TagSpecifications=[
            {
                'ResourceType': 'route-table',
                'Tags': [
                    {'Key': 'Name', 'Value': name},
                    {'Key': 'Owner', 'Value': 'Roberto Ripoll Cones'},
                    {'Key': 'tipo', 'Value': 'prueba'}
                ]
            }
        ]
    )

    return tabla
```

```python
def crear_ruta_hacia_gateway_internet(ec2, tabla_id, gateway_id):
    ec2.create_route(
        RouteTableId=tabla_id,
        DestinationCidrBlock='0.0.0.0/0',
        GatewayId=gateway_id
    )
    print(f"Ruta creada hacia el Gateway de Internet {gateway_id}")

```

1.1.6. Asociación de las subredes a la tabla personalizada

```python
def asociar_subred_a_tabla_enrutamiento(ec2, subred_id, tabla_id):
    respuesta = ec2.associate_route_table(
        RouteTableId=tabla_id,
        SubnetId=subred_id
    )

    asociacion_id = respuesta['AssociationId']

    print(f"Subred {subred_id} asociada a la tabla {tabla_id}")

    return asociacion_id

```

1.1.7. Lanzamiento EC2, asignación de IP privada y obtención de IP publica

```python
def crear_instancia(ec2, subred_id, grupo_id, ip_privada):
    instancia = ec2.run_instances(
        ImageId='ami-0866a3c8686eaeeba',
        MinCount=1,
        MaxCount=1,
        InstanceType='t2.micro',
        KeyName='vockey',
        SubnetId=subred_id,
        SecurityGroupIds=[grupo_id],
        PrivateIpAddress=ip_privada,
        TagSpecifications=[
            {
                'ResourceType': 'instance',
                'Tags': [
                    {'Key': 'Name', 'Value': 'EC2-RRC-Python'},
                    {'Key': 'Owner', 'Value': 'Roberto Ripoll Cones'},
                    {'Key': 'tipo', 'Value': 'prueba'}
                ]
            }
        ]
    )

    return instancia['Instances'][0]['InstanceId']

```

```python
def obtener_ip_instancia(ec2, instancia_id):
    ec2.get_waiter('instance_running').wait(
        InstanceIds=[instancia_id]
    )

    respuesta = ec2.describe_instances(
        InstanceIds=[instancia_id]
    )

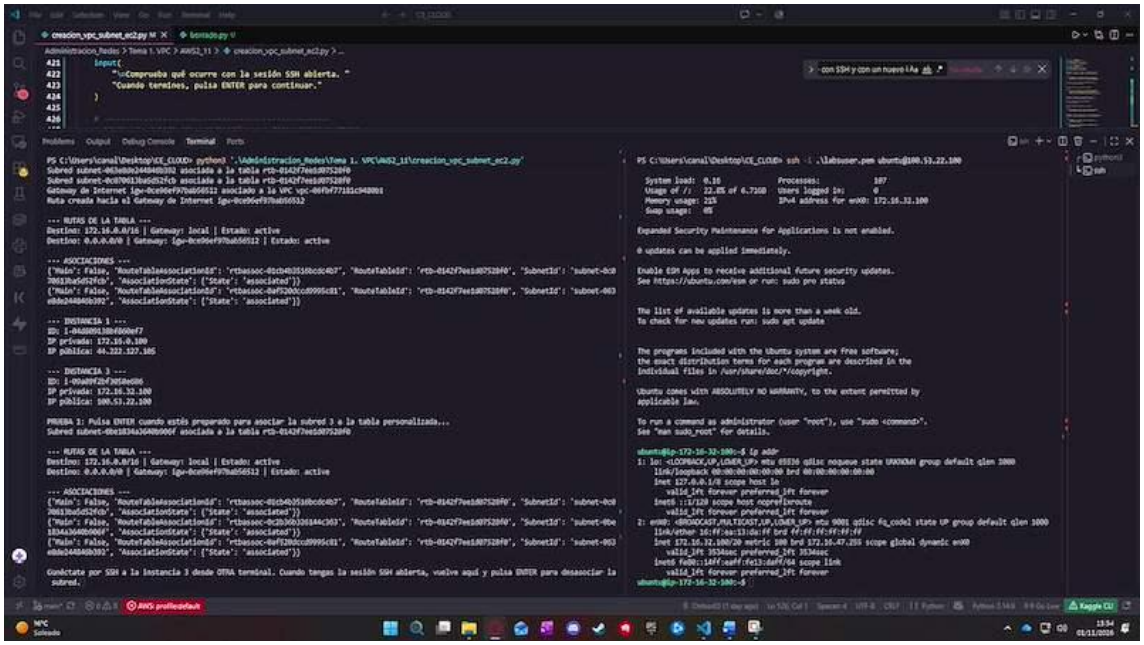
    instancia = respuesta['Reservations'][0]['Instances'][0]

    ip_privada = instancia['PrivateIpAddress']
    ip_publica = instancia.get('PublicIpAddress')

    return ip_privada, ip_publica

```

1.1.8. Orden de ejecución



Video Creación y asociación Python

Se han realizado pruebas de conexión SSH modificando las asociaciones de las subredes. Al desasociar la subred 3 de la tabla personalizada, se ha perdido la conectividad SSH, ya que la tabla principal no disponía de una ruta hacia Internet. Al volver a asociarla, se ha recuperado el acceso.

Por último, se ha desconectado el Internet Gateway, observando que la ruta predeterminada pasa al estado blackhole. Para realizar esta operación ha sido necesario detener las instancias y liberar sus IP públicas. Finalmente, se ha vuelto a asociar el gateway y se han iniciado las instancias, recuperando la conectividad con nuevas IP públicas.

1.2. Borrado

1.2.1. Búsqueda de VPC por etiquetas

```python
def obtener_vpcs_prueba(ec2):
    paginator = ec2.get_paginator('describe_vpcs')

    vpcs = []

    for pagina in paginator.paginate(
        Filters=[
            {
                'Name': 'tag:tipo',
                'Values': ['prueba']
            }
        ]
    ):
        vpcs.extend(pagina['Vpcs'])

    return vpcs
```

1.2.2. Terminación de instancias EC2

```python
def terminar_instancias(ec2, vpc_id):
    respuesta = ec2.describe_instances(
        Filters=[
            {
                'Name': 'vpc-id',
                'Values': [vpc_id]
            }
        ]
    )

    instancias_terminar = []
    instancias_esperar = []

    for reserva in respuesta['Reservations']:
        for instancia in reserva['Instances']:

            instancia_id = instancia['InstanceId']
            estado = instancia['State']['Name']

            if estado != 'terminated':
                instancias_esperar.append(instancia_id)

                if estado != 'shutting-down':
                    instancias_terminar.append(instancia_id)

    if instancias_terminar:
        ec2.terminate_instances(
            InstanceIds=instancias_terminar
        )

        print(f"Instancias terminadas: {instancias_terminar}")

    if instancias_esperar:
        ec2.get_waiter('instance_terminated').wait(
            InstanceIds=instancias_esperar
        )

        print("Todas las instancias han terminado correctamente")

    else:
        print("No hay instancias EC2 que terminar")
```

1.2.3. Eliminar recursos respetando dependencias

```python
def eliminar_tablas_enrutamiento(ec2, vpc_id):
    respuesta = ec2.describe_route_tables(
        Filters=[
            {
                'Name': 'vpc-id',
                'Values': [vpc_id]
            }
        ]
    )

    for tabla in respuesta['RouteTables']:

        tabla_id = tabla['RouteTableId']

        # No eliminamos la tabla principal de la VPC
        es_principal = any(
            asociacion.get('Main', False)
            for asociacion in tabla['Associations']
        )

        if es_principal:
            continue

        # Eliminar asociaciones explícitas que todavía existan
        for asociacion in tabla['Associations']:

            asociacion_id = asociacion.get(
                'RouteTableAssociationId'
            )

            if not asociacion.get('Main', False) and asociacion_id:
                ec2.disassociate_route_table(
                    AssociationId=asociacion_id
                )

        ec2.delete_route_table(
            RouteTableId=tabla_id
        )

        print(f"Tabla de enrutamiento {tabla_id} eliminada")

```

1.2.4. Función principal de borrado

```python
def borrar_vpc_completa(ec2, vpc_id):

    print(f"\n ELIMINANDO VPC {vpc_id} ")

    terminar_instancias(ec2, vpc_id)

    eliminar_grupos_seguridad(ec2, vpc_id)

    eliminar_subredes(ec2, vpc_id)

    eliminar_tablas_enrutamiento(ec2, vpc_id)

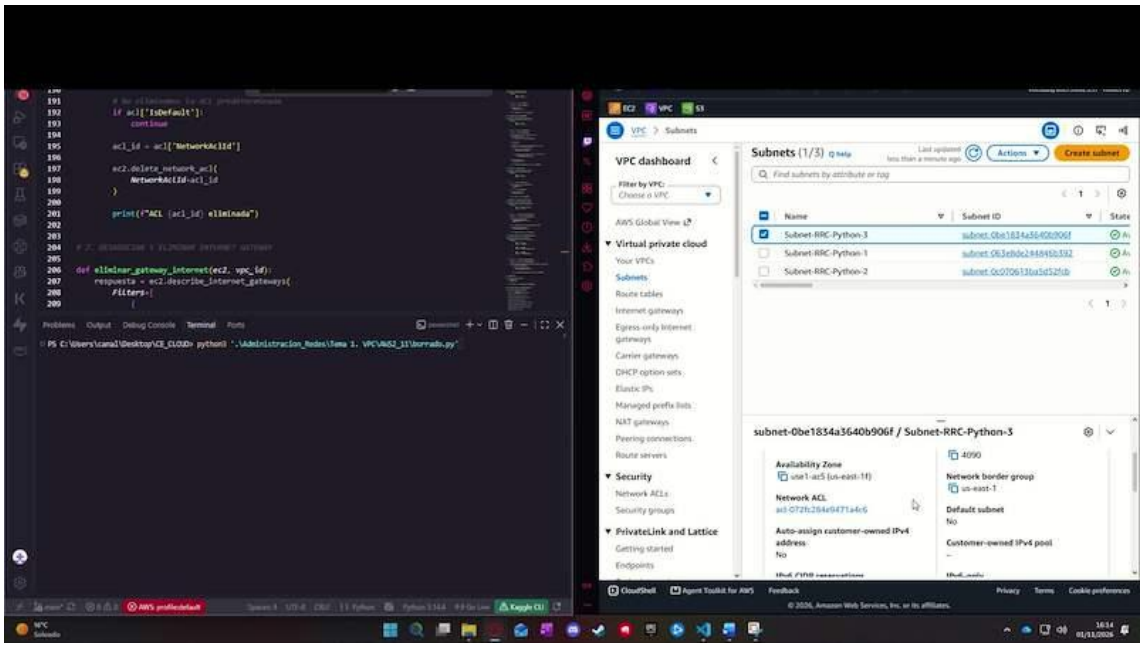
    eliminar_acl_personalizadas(ec2, vpc_id)

    eliminar_gateway_internet(ec2, vpc_id)

    eliminar_vpc(ec2, vpc_id)

    print(f" VPC {vpc_id} ELIMINADA \n")
```

1.2.5. Funcionamiento



Video borrado Python

2. Bash

2.1. Creación

2.1.1. Creación de la VPC y subredes

```
# 1. CREAR VPC Y SUBREDES
crear_vpc() {

    aws ec2 create-vpc \
        --cidr-block "$VPC_CIDR" \
        --instance-tenancy default \
        --tag-specifications "ResourceType=vpc,Tags=[{Key=Name,Value=VPC-RRC-Bash},{Key=Owner,Value=$OWNER},{Key=tipo,Value=$TIPO}]" \
        --query 'Vpc.VpcId' \
        --output text
}

```

```
crear_subred() {

    local vpc_id="$1"
    local cidr="$2"
    local nombre="$3"

    aws ec2 create-subnet \
        --vpc-id "$vpc_id" \
        --cidr-block "$cidr" \
        --tag-specifications "ResourceType=subnet,Tags=[{Key=Name,Value=$nombre},{Key=Owner,Value=$OWNER},{Key=tipo,Value=$TIPO}]" \
        --query 'Subnet.SubnetId' \
        --output text
}


```

2.1.2. Configuración del grupo de seguridad

```
crear_grupo_seguridad() {

    local vpc_id="$1"

    aws ec2 create-security-group \
        --group-name "SG-RRC-Bash" \
        --description "Grupo de seguridad para la instancia EC2" \
        --vpc-id "$vpc_id" \
        --tag-specifications "ResourceType=security-group,Tags=[{Key=Name,Value=SG-RRC-Bash},{Key=Owner,Value=$OWNER},{Key=tipo,Value=$TIPO}]" \
        --query 'GroupId' \
        --output text
}

```

```
habilitar_reglas_grupo_seguridad() {

    local grupo_id="$1"

    aws ec2 authorize-security-group-ingress \
        --group-id "$grupo_id" \
        --ip-permissions \
        'IpProtocol=tcp,FromPort=22,ToPort=22,IpRanges=[{CidrIp=0.0.0.0/0}]' \
        'IpProtocol=icmp,FromPort=-1,ToPort=-1,IpRanges=[{CidrIp=0.0.0.0/0}]'

}


```

2.1.3. Creación y asociación de la tabla de enrutamiento

```
crear_tabla_enrutamiento() {

    local vpc_id="$1"
    local nombre="$2"

    aws ec2 create-route-table \
        --vpc-id "$vpc_id" \
        --tag-specifications "ResourceType=route-table,Tags=[{Key=Name,Value=$nombre},{Key=Owner,Value=$OWNER},{Key=tipo,Value=$TIPO}]" \
        --query 'RouteTable.RouteTableId' \
        --output text
}

```

```
asociar_subred_tabla() {

    local subnet_id="$1"
    local tabla_id="$2"

    local asociacion_id

    asociacion_id=$(aws ec2 associate-route-table \
        --subnet-id "$subnet_id" \
        --route-table-id "$tabla_id" \
        --query 'AssociationId' \
        --output text)

    echo "Subred $subnet_id asociada a la tabla $tabla_id" >&2

    echo "$asociacion_id"
}

```

```
crear_ruta_gateway() {

    local tabla_id="$1"
    local gateway_id="$2"

    aws ec2 create-route \
        --route-table-id "$tabla_id" \
        --destination-cidr-block "0.0.0.0/0" \
        --gateway-id "$gateway_id"

    echo "Ruta creada hacia el Gateway de Internet $gateway_id"
}


```

2.1.4. Creación y asociación del Internet Gateway

```
crear_gateway_internet() {

    aws ec2 create-internet-gateway \
        --tag-specifications "ResourceType=internet-gateway,Tags=[{Key=Name,Value=IGW-RRC-Bash},{Key=Owner,Value=$OWNER},{Key=tipo,Value=$TIPO}]" \
        --query 'InternetGateway.InternetGatewayId' \
        --output text
}

```

```bash
asociar_gateway_internet() {

    local gateway_id="$1"
    local vpc_id="$2"

    aws ec2 attach-internet-gateway \
        --internet-gateway-id "$gateway_id" \
        --vpc-id "$vpc_id"

    echo "Gateway de Internet $gateway_id asociado a la VPC $vpc_id"
}
```

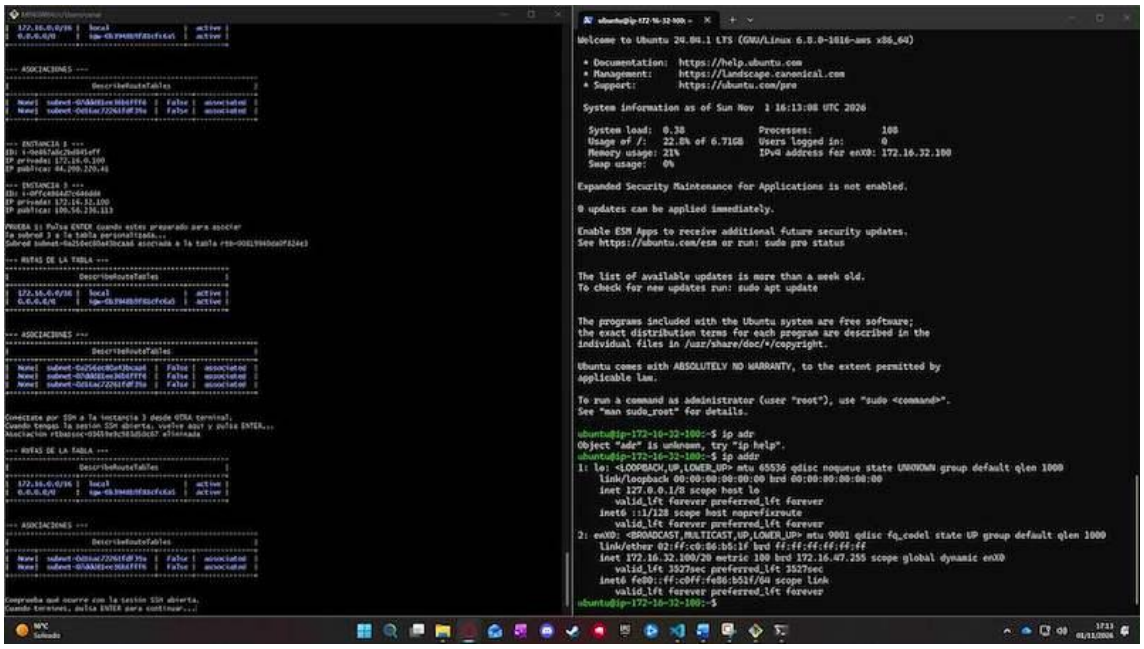
2.1.5. Creación de instancias EC2

```bash
crear_instancia() {

    local subnet_id="$1"
    local grupo_id="$2"
    local ip_privada="$3"

    aws ec2 run-instances \
        --image-id "$AMI_ID" \
        --count 1 \
        --instance-type "$INSTANCE_TYPE" \
        --key-name "$KEY_NAME" \
        --subnet-id "$subnet_id" \
        --security-group-ids "$grupo_id" \
        --private-ip-address "$ip_privada" \
        --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=EC2-RRC-Bash},{Key=Owner,Value=$OWNER},{Key=tipo,Value=$TIPO}]" \
        --query 'Instances[0].InstanceId' \
        --output text
}
```

2.1.6. Prueba ejecución



Video creación Bash

2.2. Borrado

2.2.1. Búsqueda de VPC mediante etiquetas

```
31    # 1. BUSCAR VPCS CON LA ETIQUETA TIPO=PRUEBA
32
33
34    obtener_vpcs_prueba() {
35
36        aws ec2 describe-vpcs \
37            --filters "Name=tag:tipo,Values=$TIPO" \
38            --query 'Vpcs[].VpcId' \
39            --output text
40    }
41
```

2.2.2. Terminación de instancias EC2

```
66  terminar_instancias() {
67
68      local vpc_id="$1"
69
70      echo ""
71      echo "Buscando instancias EC2 de la VPC $vpc_id..."
72
73      local instancias_esperar
74      local instancias_terminar
75
76      # Obtener instancias que todavía no estén terminadas
77
78      instancias_esperar=$(aws ec2 describe-instances \
79          --filters "Name=vpc-id,Values=$vpc_id" \
80          --query "Reservations[].Instances[?State.Name!='terminated'].InstanceId[]" \
81          --output text)
82
83      # Obtener instancias que todavía necesitan recibir terminate
84
85      instancias_terminar=$(aws ec2 describe-instances \
86          --filters "Name=vpc-id,Values=$vpc_id" \
87          --query "Reservations[].Instances[?State.Name!='terminated' && State.Name!='shutting-down'].InstanceId[]" \
88          --output text)
89
90      if [[ -n "$instancias_terminar" && "$instancias_terminar" != "None" ]]; then
91
92          aws ec2 terminate-instances \
93              --instance-ids $instancias_terminar
94
95          echo "Instancias terminadas: $instancias_terminar"
96
97      fi
98
99      if [[ -n "$instancias_esperar" && "$instancias_esperar" != "None" ]]; then
100
101         aws ec2 wait instance-terminated \
102             --instance-ids $instancias_esperar
103
104         echo "Todas las instancias han terminado correctamente."
105
106     else
107
108         echo "No hay instancias EC2 que terminar."
109
110     fi
111
112 }
```

2.2.3. Eliminación de tablas de enrutamiento

```
197  eliminar_tablas_enrutamiento() {
204      local tablas
205
206      tablas=$(aws ec2 describe-route-tables \
207          --filters "Name=vpc-id,Values=$vpc_id" \
208          --query 'RouteTables[].RouteTableId' \
209          --output text)
210
211      if [[ -z "$tablas" || "$tablas" == "None" ]]; then
212
213          echo "No hay tablas de enrutamiento."
214
215          return
216
217      fi
218
219      for tabla_id in $tablas; do
220
221          local principal
222
223          principal=$(aws ec2 describe-route-tables \
224              --route-table-ids "$tabla_id" \
225              --query 'RouteTables[0].Associations[?Main==`true`].Main | [0]' \
226              --output text)
227
228          if [[ "$principal" == "True" ]]; then
229
230              echo "Tabla principal $tabla_id: se conserva."
231
232              continue
233
234          fi
235
236          echo "Eliminando tabla personalizada $tabla_id..."
237
238          local asociaciones
239
240          asociaciones=$(aws ec2 describe-route-tables \
241              --route-table-ids "$tabla_id" \
242              --query 'RouteTables[0].Associations[?SubnetId!=null].RouteTableAssociationId' \
243              --output text)
244
245          if [[ -n "$asociaciones" && "$asociaciones" != "None" ]]; then
246
247              for asociacion_id in $asociaciones; do
248
249                  aws ec2 disassociate-route-table \
250                      --association-id "$asociacion_id"
251
252                  echo "Asociación $asociacion_id eliminada."
253
254              done
255
256          fi
257
258          aws ec2 delete-route-table \
259              --route-table-id "$tabla_id"
260
261          echo "Tabla $tabla_id eliminada."
262
263      done
264
265  }
```

2.2.4. Eliminación de Internet Gateway

```
312  eliminar_gateway_internet() {
313
314      local vpc_id="$1"
315
316      echo ""
317      echo "Buscando Internet Gateway asociados..."
318
319      local gateways
320
321      gateways=$(aws ec2 describe-internet-gateways \
322          --filters "Name=attachment.vpc-id,Values=$vpc_id" \
323          --query 'InternetGateways[].InternetGatewayId' \
324          --output text)
325
326      if [[ -z "$gateways" || "$gateways" == "None" ]]; then
327
328          echo "No hay Internet Gateway asociados."
329
330          return
331
332      fi
333
334      for gateway_id in $gateways; do
335
336          # Desasociar Internet Gateway
337
338          aws ec2 detach-internet-gateway \
339              --internet-gateway-id "$gateway_id" \
340              --vpc-id "$vpc_id"
341
342          echo "Gateway $gateway_id desasociado de la VPC."
343
344          # Eliminar Internet Gateway
345
346          aws ec2 delete-internet-gateway \
347              --internet-gateway-id "$gateway_id"
348
349          echo "Gateway $gateway_id eliminado."
350
351      done
352
353  }
354
```

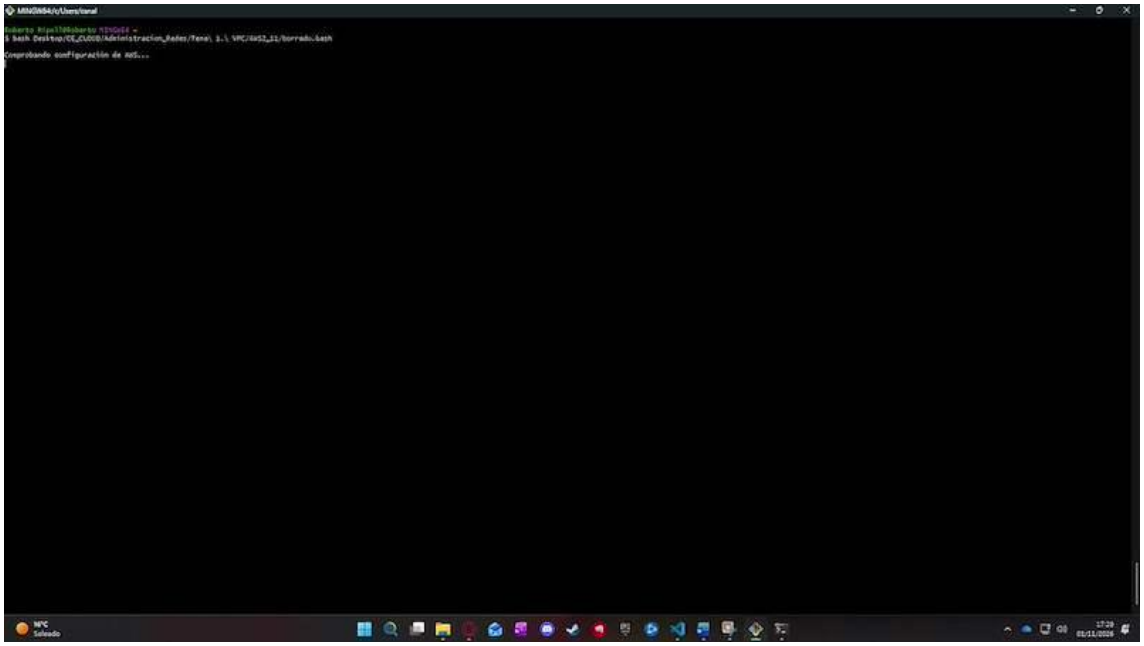
2.2.5. Gestión de interfaces de red (ENI)

```
357  eliminar_interfaces_red() {
358
359      local vpc_id="$1"
360
361      echo ""
362      echo "Buscando interfaces de red restantes..."
363
364      local interfaces
365
366      interfaces=$(aws ec2 describe-network-interfaces \
367          --filters "Name=vpc-id,Values=$vpc_id" \
368          --query 'NetworkInterfaces[].NetworkInterfaceId' \
369          --output text)
370
371      if [[ -z "$interfaces" || "$interfaces" == "None" ]]; then
372
373          echo "No hay interfaces de red pendientes."
374
375          return
376
377      fi
378
379      for interfaz_id in $interfaces; do
380
381          local administrada
382          local estado
383          local descripcion
384
385          administrada=$(aws ec2 describe-network-interfaces \
386              --network-interface-ids "$interfaz_id" \
387              --query 'NetworkInterfaces[0].RequesterManaged' \
388              --output text)
389
390          estado=$(aws ec2 describe-network-interfaces \
391              --network-interface-ids "$interfaz_id" \
392              --query 'NetworkInterfaces[0].Status' \
393              --output text)
394
395          descripcion=$(aws ec2 describe-network-interfaces \
396              --network-interface-ids "$interfaz_id" \
397              --query 'NetworkInterfaces[0].Description' \
398              --output text)
399
400          echo "Interfaz: $interfaz_id"
401          echo "Estado: $estado"
402          echo "Administrada por AWS: $administrada"
403          echo "Descripción: $descripcion"
404
405          if [[ "$administrada" == "True" ]]; then
406
407              echo "No se puede eliminar directamente: depende de otro servicio."
408
409          elif [[ "$estado" == "available" ]]; then
410
411              aws ec2 delete-network-interface \
412                  --network-interface-id "$interfaz_id"
413
414              echo "Interfaz $interfaz_id eliminada."
415
416          else
417
418              echo "La interfaz no está disponible para su eliminación."
419
420          fi
421
422      done
423
424  }
```

2.2.6. Función principal de borrado y confirmación

```
484  borrar_vpc_completa() {
485
486      local vpc_id="$1"
487
488      echo ""
489      echo "ELIMINANDO VPC $vpc_id"
490
491      terminar_instancias "$vpc_id"
492
493      eliminar_grupos_seguridad "$vpc_id"
494
495      eliminar_subredes "$vpc_id"
496
497      eliminar_tablas_enrutamiento "$vpc_id"
498
499      eliminar_acl_personalizadas "$vpc_id"
500
501      eliminar_gateway_internet "$vpc_id"
502
503      eliminar_interfaces_red "$vpc_id"
504
505      eliminar_vpc "$vpc_id"
506
507      echo ""
508      echo "VPC $vpc_id ELIMINADA"
509      echo ""
510
511  }
```

2.2.7. Ejecución del script



Video borrado Bash